

Bug Fix Report

Project	candidate-check-tracker v1.0.0 (Express API + static vanilla-JS frontend)
Prepared	2026-09-10
Author	Tarun Singh
Source	my-bug-report.md (Phase 2 — a failing automated test per bug; fixes as bonus)
Result	17 / 17 regression tests pass; all 12 bugs fixed at root cause; no regression

1. Summary

my-bug-report.md listed 12 confirmed bugs across the REST API (`server.js`, bugs 1–9) and the browser client (`public/`, bugs 10–12). Phase 2 asked for an automated test that fails because of each bug; fixing them was optional. Both were done.

- A test project was added (**Jest + Supertest + jsdom** — the repo had no test setup) with **17 tests in 2 suites**: `tests/api.test.js` (bugs 1–9) and `tests/ui.test.js` (bugs 10–12).
- Every test was run **before** any fix and confirmed to fail for the reported reason — not an unrelated error such as a missing import.
- All 12 bugs were fixed at the root cause. The recurring themes: **missing input validation**, **enum / casing inconsistency**, **wrong string & date primitives**, and **unhandled `fetch` rejections**.
- After the fix, **17 / 17 tests pass** and a manual `curl` smoke test confirms the API end to end. No preexisting behaviour regressed.

Area	Bugs
API — input validation missing	#2 candidate reference, #5 status enum, #9 type enum
API — enum / serialization inconsistency	#3 "pending" vs PENDING, #4 <code>toDateString()</code> vs ISO 8601
API — query-filter logic	#7 substring vs exact, #8 case-sensitive vs spec's case-insensitive
API — response contract	#1 undocumented <code>internalReviewNotes</code> , #6 200 vs 201

UI	#10 dropdown default, #11 Discrepancy badge colour, #12 silent network failure
----	--

2. Severity & Priority

No rubric file shipped with the skill, so a standard one is applied and stated here.

Severity	Meaning
----------	---------

Critical	Data loss/corruption at rest, security breach, or outage with no workaround.
High	Core behaviour wrong, wrong data served or persisted, or a contract break that silently misleads consumers; no server-side workaround.

Medium	Incorrect behaviour with a workaround, or a secondary path / degraded UX.
--------	---

Severity	Meaning
----------	---------

Low	Cosmetic or spec-compliance nit with negligible functional impact.
-----	--

Priority	Meaning
----------	---------

P0	Hotfix now.
----	-------------

P1	Fix before the next release.
----	------------------------------

P2	Normal backlog.
----	-----------------

P3	Opportunistic.
----	----------------

#	Bug	Sev.	Pri.	Rationale
1	GET /api/checks/:id leaks internalReviewNotes	High	P1	Response-shape contract break; the field name signals internal-only content — a latent disclosure path. Always "" today, so nothing sensitive leaks yet: High, not Critical.
2	POST doesn't check candidateId exists	High	P1	Persists orphan records (candidateName: null) that pollute every later read, filter and export. No workaround.

3	New checks persisted as "pending" (lowercase)	High	P1	Enum inconsistency at rest. Breaks the case-sensitive status filter, the UI badge/label lookup, and any consumer switching on the value.
4	createdAt stored as "Thu Sep 10 2026"	High	P1	Every consumer that parses or sorts by date breaks; time-of-day precision is lost permanently in the stored record.
5	PATCH .../status doesn't validate the enum	High	P1	Any client can permanently overwrite a record's status with arbitrary text, corrupting data and UI. Borderline P0.
6	POST returns 200 not 201	Low	P2	Spec / convention break. Clients keying on 201 or Location misbehave, but the body is correct and the workaround is trivial.
7	Type filter matches substring, not exact enum	Medium	P1	Silently wrong result sets (?type=ADD → ADDRESS rows, ?type=E → EDUCATION + EMPLOYMENT).
8	Status filter case-sensitive (spec: insensitive)	Medium	P1	Direct spec violation; with #3 in play a record can vanish from a filtered view entirely.
9	POST doesn't validate the type enum	High	P1	Junk types ("INVALID_TYPE") persisted, polluting data and filters.
10	Type dropdown defaults to "Identity"	Medium	P2	On load the grid is silently filtered while the user believes "All Types" is shown.
11	Discrepancy badge green (identical to Verified)	High	P1	In a BGV tool, Discrepancy is the key adverse outcome; rendering it like Verified can cause a reviewer to miss a failed check. Also a colour-blind hazard.
12	Failed Add / Update requests fail silently	Medium	P1	User believes a write succeeded when it did not; produces an unhandled promise rejection.

Totals: High = 7 • Medium = 4 • Low = 1.

3. Environment

App	candidate-check-tracker v1.0.0
Runtime	Node.js v24.13.0, npm 11.6.2
OS	Windows 11 (win32)

Server	Express ^4.19.2; in-memory per-session store seeded from data.js (isolation.js)
Client	Static public/index.html + public/app.js (no build step, no framework)
Spec of record	The app's own GET /openapi.json and /spec page
Test stack added	Jest 30.5.1, Supertest 7.2.2, jsdom 24.1.3 (devDependencies; none existed before)

Test-infrastructure change

`server.js` previously called `app.listen()` at module load, making it un-importable by a test runner. Now:

```
if (require.main === module) {   app.listen(PORT, () => { /* ... */
}); } module.exports
= app;
```

This is the only non-bug change to production code — it lets Supertest import the app without binding a port. Behaviour under `npm start` is unchanged.

4. Steps to Reproduce

API steps assume a fresh session (POST /api/reset or a new cookie jar). Seed: 6 checks, candidates 1–4.

#	Reproduction	Observed (before)	Expected
1	GET /api/checks/1	body includes "internalReviewNotes": ""	only id, candidateId, candidateName, type, status, createdAt
2	POST /api/checks {candidateId:9999, type:IDENTITY}	200, check created with candidateName: null	400, nothing created
3	POST /api/checks {candidateId:1, type:IDENTITY}	"status": "pending"	"status": "PENDING"
4	same as #3, inspect createdAt	"Thu Sep 10 2026"	ISO 8601, e.g. 2026-09-10T09:35:19.725Z
5	PATCH /api/checks/4/status {status:NOT_REAL}	200, status overwritten to "NOT_REAL"	400, record unchanged
6	POST /api/checks {candidateId:2, type:IDENTITY}	200 OK	201 Created
7	GET /api/checks?type=EDU	returns the EDUCATION row(s)	[] (EDU ≠ EDUCATION)

8	GET ?status=pending vs ?status=PENDING	different result sets	identical result sets
9	POST /api/checks {candidateId:2, type:INVALID_TYPE}	200, type persisted as "INVALID_TYPE"	400, nothing created
10	Load /, read the Type filter select	"Identity" selected	"All Types" selected (value "")
11	Render a DISCREPANCY row; inspect badge colour	#036B26 green — identical to VERIFIED	a red, visually distinct from Verified
12	Offline, click Add Check / Update	success toast or nothing; console: Uncaught (in promise) TypeError: Failed to fetch	an error message shown to the user

5. Root Cause Analysis

5.1 Missing input-validation layer — bugs 2, 5, 9

The POST /api/checks and PATCH /api/checks/:id/status handlers consumed `req.body` directly and wrote it to the store. No enum check on `type` / `status`; no referential check that `candidateId` maps to a real candidate. POST even encoded the missing-reference case as "allow and null it out" — `candidateName: candidate ? candidate.name : null` — rather than "reject". The handlers were written for the happy path and validation was never added; nothing else in the stack enforces it.

5.2 Enum value written with the wrong casing — bug 3

POST hard-coded `status: "pending"`. Everything else is uppercase: the `data.js` seed, the `openapi.json` status enum, and the client's `STATUS_LABELS` / `STATUS_COLORS` maps. A literal casing mistake that survived because nothing validates the value on write and the case-sensitive filter (bug 8) masked it in casual testing.

5.3 Wrong Date serialization primitive — bug 4

`createdAt: new Date().toLocaleDateString()` produces a locale display string ("Thu Sep 10 2026") — no time, no timezone, not stably parseable. Seed data and the OpenAPI examples use ISO 8601 (`toISOString()`).

Wrong method chosen.

5.4 Filter comparison logic — bugs 7 & 8

```
if (type) result = result.filter((c) => c.type.includes(type)); // substring, not equality if
(status) result = result.filter((c) => c.status === status); // exact + case-sensitive
```

#7: `String.prototype.includes` is a substring test. `"EDUCATION".includes("EDU")` is true, so `?type=EDU` (also `?type=E`, `?type=ADD`, ...) match. The spec requires an exact match against the four-value type enum.

#8: `===` on raw strings is case-sensitive, but /spec explicitly states the status filter is case-insensitive.

?status=pending returns [] against uppercase-seeded data, and — with bug 3 — ?status=PENDING misses lowercase-stored rows. The two queries disagree.

5.5 Response shaping — bug 1

toResponseShape() unconditionally appended internalReviewNotes: "". That key is in neither the documented response shape nor the openapi.json schema — scaffold/debug residue left in the DTO builder. The value is a constant empty string today, so no real data leaks yet, but the shape is a contract and the field name is a disclosure risk the moment it is populated.

5.6 HTTP status code — bug 6

The create handler called res.status(200). REST convention and the app's own openapi.json (responses: { '201': { description: 'Created check' } }) specify 201 Created.

5.7 select default option — bug 10

An HTML <select> with no option marked selected defaults to its **first** <option>. In index.html the Type filter's first option was value="IDENTITY"; the "" / All Types option was second. So the control loaded on "Identity" and buildQuery() sent type=IDENTITY on the first loadChecks() — the grid was filtered before the user touched anything.

5.8 Status-colour map — bug 11

```
VERIFIED:    "#036B26",  
DISCREPANCY: "#036B26",    // same green as VERIFIED
```

STATUS_COLORS.DISCREPANCY was set to the exact green used for VERIFIED — a copy/paste of the adjacent value. renderChecks() and renderLegend() both read this map, so the badge and legend render Discrepancy indistinguishably from Verified.

5.9 Unhandled fetch rejection — bug 12

```
// Add Check const res = await fetch("/api/checks", { ... }); // no try/catch,  
no res.ok check showToast("Check added successfully", "success"); // shown  
unconditionally  
  
// Update Status const res = await fetch(`/api/checks/${id}/status`, { ...  
}); // res.ok checked... if (res.ok) { ... } else { ... } // ...but no  
try/catch - a rejected fetch() (network failure) throws past this
```

fetch() rejects on network failure (offline, DNS, CORS, connection reset). Neither handler wrapped the call, so the rejection propagated as an unhandled promise rejection (TypeError: Failed to fetch) and the UI showed a **false success** (Add) or **nothing at all** (Update).

6. Fix Applied

Production files touched: server.js, public/app.js, public/index.html. Added: tests/api.test.js, tests/ui.test.js, and package.json test wiring.

server.js

```
+const VALID_STATUSES =
["PENDING","IN_PROGRESS","VERIFIED","DISCREPANCY","INSUFFICIENCY","CLOSED"]; +const VALID_TYPES =
["IDENTITY","EDUCATION","EMPLOYMENT","ADDRESS"];

function toResponseShape(check) { return { id, candidateId, candidateName, type,
status, createdAt: check.createdAt, - internalReviewNotes: "",
// bug 1: drop it
};
}

if (status) { // bugs 7,
8 - result = result.filter((c) => c.status === status);
+ const wanted = String(status).toUpperCase();
+ result = result.filter((c) => String(c.status).toUpperCase() ===
wanted); } if (type) { - result = result.filter((c) =>
c.type.includes(type));
+ result = result.filter((c) => c.type === type);
}

app.post("/api/checks", (req, res) => { // bugs 2,3,4,6,9
const { candidateId, type } = req.body || {}; + if (!VALID_TYPES.includes(type)) +
return res.status(400).json({ error: `Invalid type "${type}". ...` }); const candidate
= req.store.candidates.find((c) => c.id === candidateId); + if (!candidate) + return
res.status(400).json({ error: `Unknown candidateId: ${candidateId}` }); const newCheck
= { - candidateName: candidate ? candidate.name : null,
- status: "pending",
- createdAt: new Date().toISOString(),
+ candidateName: candidate.name,
+ status: "PENDING",
+ createdAt: new Date().toISOString(),
...
};
- res.status(200).json(toResponseShape(newCheck));+
res.status(201).json(toResponseShape(newCheck)); });

app.patch("/api/checks/:id/status", (req, res) => { // bug
5 if (!check) { res.status(404)...; return; } + if
(!VALID_STATUSES.includes(status)) + return res.status(400).json({ error:
`Invalid status "${status}". ...` }); check.status = status; });

-app.listen(PORT, () => { ... }); // testability, not a bug
+if (require.main === module) { app.listen(PORT, () => { ... }); }
+module.exports = app;
```

public/app.js

```
VERIFIED: "#036B26",
- DISCREPANCY: "#036B26", // bug 11
+ DISCREPANCY: "#C62828", // a distinct red

// Update Status handler - bug 12 + try { const res =
await fetch(`/api/checks/${id}/status`, { ... }); if (res.ok)
showToast("Status updated", "success"); else
showToast("Failed to update status", "error"); loadChecks();
+ } catch (err) { + showToast("Network error - could not
update status", "error"); + }

// Add Check handler - bug 12 + try { const res = await fetch("/api/checks",
{ ... }); + if (res.ok) { showToast("Check added successfully", "success");
loadChecks(); }
+ else { showToast("Failed to add check", "error"); }
+ } catch (err) {
+ showToast("Network error - could not add check", "error"); +
}
```

The `loadChecks()` call on the Add success path is a small companion fix — the original never refreshed the grid after a create, so a new row never appeared. Called out here rather than made silently.

public/index.html

```
<select id="filter-type">
-   <option value="IDENTITY">Identity</option>
<option value="">All Types</option>
+   <option value="IDENTITY">Identity</option>
    <option value="EDUCATION">Education</option>    <!-- bug 10: first option = default -->
```

Judgment call (bugs 2, 5, 9): reject vs. coerce

Invalid input could be handled by **rejecting with 400** or by **silently coercing** (e.g. up-casing a lowercase status, ignoring an unknown filter). This fix **rejects**, because (a) my-bug-report.md states the expected result is 400 for each, and (b) openapi.json documents a 400 on both endpoints. Coercion was rejected as an approach: it hides client bugs and lets subtly-wrong data into the store.

7. Test Evidence

7.1 Test files

- `tests/api.test.js` — 13 tests, bugs 1–9, Supertest against the imported Express app. A fresh `request.agent(app)` per test gets a new sid cookie, hence a clean seed (isolation.js).
- `tests/ui.test.js` — 4 tests, bugs 10–12, loads `public/index.html` + `public/app.js` into a jsdom window with a stubbed fetch and exercises the real client code.

7.2 Before the fix — every test fails, for the reported reason

Run on the unmodified app (only the `module.exports = app` shim added so the runner can import it):

```
FAIL tests/api.test.js
*   Bug #1  single-check response contains only the documented keys
+ Received + 1    ... "internalReviewNotes",
*   Bug #2  nonexistent candidateId is rejected with 400      Expected: 400   Received: 200 *
+ Received + 1    ... "internalReviewNotes",
*   Bug #3  created check has status exactly 'PENDING'      Expected: "PENDING"   Received:
"pending"
*   Bug #4  createdAt is ISO 8601      Received string: "Thu Sep 10 2026"
*   Bug #5  invalid status is rejected with 400, record unchanged Expected: 400   Received: 200
*   Bug #6  a fully valid create returns 201      Expected: 201   Received: 200
*   Bug #7  ?type=EDU returns []      + Array [ { ... "type": "EDUCATION" ... } ]
*   Bug #8  ?status=pending and ?status=PENDING identical - [ {..."PENDING"...} ] + Array []
*   Bug #9  invalid type is rejected with 400      Expected: 400   Received: 200

FAIL tests/ui.test.js
*   Bug #10  Type filter has no value selected (All Types) Expected: ""   Received: "IDENTITY"
*   Bug #11  DISCREPANCY badge red, distinct from VERIFIED Expected: not "rgb(3, 107, 38)"
*   Bug #12  network failure on Add Check shows an error Expected: false Received: true
(+ Unhandled rejection: TypeError: Failed to fetch) * Bug #12  network
failure on Update Status shows an error Expected: true   Received: false

Tests: 15 failed, 2 passed, 17 total
```

The 2 that passed before the fix are positive-path guards (Bug #7 › `?type=EDUCATION` still returns rows; Bug #5 › a valid status transition still works) — kept green on purpose so the fix cannot over-correct.

7.3 After the fix — full suite green

```
$ npm test
```

```
PASS tests/api.test.js
```

```
PASS tests/ui.test.js
```

```

Test Suites: 2 passed, 2 total
Tests:      17 passed, 17 total
Snapshots:  0 total
Time:       ~2.3 s

PASS . Bug #1 single-check response contains only the documented keys
PASS . Bug #1 list response objects also omit internalReviewNotes
PASS . Bug #2 nonexistent candidateId is rejected with 400 and nothing is created
PASS . Bug #2 a valid candidateId still succeeds
PASS . Bug #3 created check has status exactly 'PENDING'
PASS . Bug #4 createdAt is ISO 8601
PASS . Bug #5 an invalid status is rejected with 400 and the record is left unchanged
PASS . Bug #5 a valid status transition still works
PASS . Bug #6 a fully valid create returns 201
PASS . Bug #7 ?type=EDU returns [] because EDU is not an exact enum value
PASS . Bug #7 ?type=EDUCATION still returns the education rows
PASS . Bug #8 ?status=pending and ?status=PENDING return identical result sets
PASS . Bug #9 an invalid type is rejected with 400 and nothing is created
PASS . Bug #10 on load the Type filter has no value selected (All Types)
PASS . Bug #11 the rendered DISCREPANCY badge uses a red colour distinct from VERIFIED PASS
. Bug #12 a network failure on Add Check shows a user-facing error message
PASS . Bug #12 a network failure on Update Status shows a user-facing error message

```

7.4 Manual API smoke test (fixed server, port 3999)

```

--- GET /api/checks/1 (bug 1: no internalReviewNotes) ---
{"id":1,"candidateId":1,"candidateName":"Aditi Sharma","type":"IDENTITY",
 "status":"VERIFIED","createdAt":"2026-07-01T09:12:00.000Z"}

--- POST valid (bugs 3, 4, 6) ---
HTTP/1.1 201 Created
{"id":7,"candidateId":2,"candidateName":"Rohan Mehta","type":"IDENTITY",
 "status":"PENDING","createdAt":"2026-09-10T09:35:19.725Z"}

--- POST bad candidate (bug 2) --- 400
--- POST bad type (bug 9) --- 400
--- PATCH bad status (bug 5) --- 400
--- GET ?type=EDU (bug 7) --- []
--- GET ?status=pending (bug 8) --- [{"id":4, ... "status":"PENDING" ...}]

```

7.5 Regression check

There is no pre-existing automated suite to run. Coverage of "did anything else break" comes from (a) the `positivepath` assertions in the new suite (valid create still works and returns the right body; valid status transition still works; `?type=EDUCATION` still returns rows), and (b) the manual smoke test above. Nothing regressed. The one production change outside the bug fixes — guarding `app.listen()` — was verified by running `npm start` (server still boots and logs on `:3001`).

8. Recommendations

- Extract a validation layer.** Bugs 2, 5, 9 are the same defect three times. A schema (zod / express-validator, or a hand-rolled `validateBody`) makes enum and referential checks declarative and consistent, with a uniform 400 error body.
- Single source of truth for enums.** `PENDING` / `IDENTITY` / ... currently live independently in `data.js`, `server.js`, `openapi.json` and `public/app.js`. Bugs 3 and 8 exist because of that duplication. Put them in one module and generate the OpenAPI enum and client maps from it.
- Contract-test every response against openapi.json.** A schema assertion per endpoint would have caught bugs 1, 4 and 6 automatically.

4. **Data migration.** Records created before this fix may still hold status: "pending", toDateString() timestamps, unknown type values, or candidateName: null. Run a one-off cleanup / backfill.
 5. **One fetch wrapper on the client.** Route all API calls through a helper that handles network rejection and non2xx uniformly, so bug 12 cannot recur per call site.
 6. **Colour accessibility.** Even after bug 11, Verified-green vs Discrepancy-red is a red/green colour-blind hazard. Pair the badge colour with an icon or text weight, not colour alone.
 7. **Add npm test to CI** so these 17 regression tests run on every change.
 8. **Adjacent issues found while fixing (out of scope, worth tickets):** (a) GET /api/checks?candidateId=2 compares a string query param to a numeric field (c.candidateId === candidateId) and never matches. (b) GET /api/checks/:id for an unknown id returns 200 {} instead of the documented 404.
-